Shop, book, exchange, and cancel Split Tickets in SilverCore

Portfolio Context

Role: Lead Technical Writer
Context: This implementation guide explains how to handle "Split Ticketing" in our booking API. Split Ticketing is a uniquely complex UK rail business logic where a customer takes one continuous journey but holds multiple tickets to circumvent high through-fares. Managing partial refunds, order exchanges, and payment statuses across disconnected logical reservations required tight coordination with backend developers to document accurately.

Intro

For the UK market only, SilverCore provides Partners the possibility of offering customers cheaper tickets for their journeys through Split Ticketing. Split Ticketing works by finding combinations of tickets that together offer the same journey but at a lower price than a regular ticket.

The following diagram shows how a journey from Manchester Piccadilly to Sheffield is split into two or more tickets. Please note that the prices shown are purely representative and serve only to explain the concept of Split Ticketing.

Supported markets

Split Ticketing is supported for all RDG carriers.

Info

Please raise a JIRA ticket to enable Split Ticketing for your context.

Working with split ticketing in SilverCore

Please observe the following points when implementing Split Ticketing:

- SilverCore supports Advance, Off-peak, and Anytime fares but does not support Split Ticketing on "open return" searches. (SilverCore will return an error when shopping for Split Tickets with `includeReturnFares` as a request parameter.)
- All Split Tickets on a booking order must be fulfilled using the same method. This restriction may lead to a lot of purchases defaulting to TOD only.
- To prevent fraud, canceling individual Split Tickets on an order is not supported. The entire order must be canceled (full cancel), or all the Ticketable Fares for a given Leg Solution must be canceled together (partial cancel).
- Split Tickets with Advance fares cannot be canceled in SilverCore.
- Orders with a mixture of Advance and Anytime fares cannot be canceled in SilverCore.
- Split Ticket exchanges are supported for one-way journeys that contain Advance fares (either exclusively or mixed with Walk-up fares). If the first train on the journey has already departed, the exchange is not permitted.

> **Warning**
>
> Round-trip Split Ticket bookings cannot currently be exchanged.

Ancillary services

SilverCore supports seat reservations but NOT bike reservations, Travel Cards, or PlusBus tickets for Split Ticket bookings. Unsupported ancillary services will not be returned in the validateBookingRecordInformationResponse.

Please refer to our How-To guide for more information about working with ancillary services in SilverCore.

Seat reservations

On those journeys where passengers are not required to change services, SilverCore will attempt to allocate the same seat to ensure that passengers are not required to change carriage or seat while still on the same train.

Shopping and booking flow in SilverCore for Split Tickets

Shopping for Split Ticket fares

Shopping for Split Ticket fares mirrors the typical Point-To-Point shopping flow, with the addition of several Split Ticket-specific request elements.

SilverCore assesses each requested journey for Split Ticketing by searching for the lowest combination of Advance, Off-peak, and Anytime fares that will work for that journey. Each combination is returned in a distinct `summatedFare` element in the shopping response.

pointToPointShoppingRequest

To shop for Split Ticket fares, perform a pointToPointShoppingRequest and pass in the mandatory split ticket data in the `splitTicketing` container element:

- `maxSplits`
- `maxSummatedFares`
- `automaticSplitAtCallingStations` OR `automaticSplitAtInterchanges`

Together with any optional split ticket data required:

- `summatedFareFilter`
- `includeFullSummatedFaresDetails`

Example pointToPointShoppingRequest for a one-way journey between Manchester Piccadilly and Sheffield:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
    xmlns:com= "http://railgds.net/ws/commontypes"
    xmlns:book="http://railgds.net/ws/booking"
    xmlns:shop="http://railgds.net/ws/shopping"
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Header/>
    <soap:Body>
        <shop:pointToPointShoppingRequest version="2.0">
            <com:context>
                <com:distributorCode>DEMO</com:distributorCode>
                <com:pointOfSaleCode>GB</com:pointOfSaleCode>
                <com:channelCode>IOS</com:channelCode>
            </com:context>
            <shop:pointToPointShoppingQuery>
                <shop:travelPointPairs>
                    <shop:travelPointPair>
                        <com:originTravelPoint
type="STATION">GBQQM</com:originTravelPoint>
```

```
                        <com:destinationTravelPoint
type="STATION">GBSHF</com:destinationTravelPoint>
                        <com:departureDateTimeWindow>
                            <com:date>2022-10-31</com:date>
                            <com:time>12:00:00</com:time>
                        </com:departureDateTimeWindow>
                    </shop:travelPointPair>
                </shop:travelPointPairs>
                <shop:passengerSpecs>
                    <shop:passengerSpec>
                        <com:age>26</com:age>
                    </shop:passengerSpec>
                </shop:passengerSpecs>
                <shop:fareQualifiers>
                    <shop:fareQualifier type="DISCOUNT_CARD">
                        <com:program>UK_SENIOR</com:program>
                    </shop:fareQualifier>
                </shop:fareQualifiers>
❶               <shop:splitTicketing maxSplits="2" maxSummatedFares="3">
                    <com:automaticSplitAtCallingStations/>
                    <com:summatedFareFilters>
❷                       <com:summatedFareFilter fareCategory="All"
fareClass="All"/>
                    </com:summatedFareFilters>
                </shop:splitTicketing>
            </shop:pointToPointShoppingQuery>
        </shop:pointToPointShoppingRequest>
    </soap:Body>
</soap:Envelope>
```

❶ Return a maximum of three summated fares per Leg Solution and split the fare into a maximum of three tickets (two split points) in each direction. ❷ Don't restrict the Split Ticket fares by class or category.

Request elements

Refer to the following descriptions for the details of each request element and attribute related to Split Ticketing in SilverCore.

> pointToPointShoppingRequest pointToPointShoppingQuery **shop:splitTicketing**

A container element that defines what split ticketing solutions are returned in the response. The element must contain exactly one of `automaticSplitAtCallingStations` or `automaticSplitAtInterchanges`.

The `splitTicketing` element has the following attributes:

Attribute	Description
`maxSplits` **(Required)**	The maximum number of times a ticket may be split. Partners are advised to set a value of 2 or less. For example, if `maxSplits=2`, then each Split Ticketing solution (returned in a distinct `summatedFare` element) will be split at no more than two calling stations or interchanges.
`maxSummatedFares` **(Required)**	The maximum number of Split Ticketing fares returned per Leg Solution for each `summatedFareFilter` included in your request. The total number of Split Ticketing fares returned for a Leg Solution can be up to `maxSummatedFares` multiplied by the number of `<summatedFareFilter>` elements. For example, with `maxSummatedFares="3"` and the following two filters: 1. `<summatedFareFilter fareCategory="All" fareClass="All"/>` (Any class/category) 2. `<summatedFareFilter fareCategory="All" fareClass="First"/>` (First class only) SilverCore will return up to **6** summated fares per Leg Solution: - The 3 cheapest options across all classes/categories. - The 3 cheapest First Class options. *(Duplicate fares found by multiple filters are removed, so the actual count may be lower if the cheapest First Class fares are also among the cheapest overall fares.)*

Example:

```
<shop:splitTicketing maxSummatedFares='1' maxSplits='2'>
```

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing
> **com:automaticSplitAtCallingStations**

Split fares at any calling station along the journey route. In the context of Split Ticketing, a calling station can also be an interchange where the passenger must change trains to

continue their journey.

You **must** include either this element OR `<automaticSplitAtInterchanges>`. Cannot be used together.

Example:

```
<com:automaticSplitAtCallingStations/>
```

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing
> **com:automaticSplitAtInterchanges**

Split fares at any interchange along the journey route. In the context of split ticketing an interchange is where the passenger must change trains to continue their journey.

You **must** include either this element OR `<automaticSplitAtCallingStations>`. Cannot be used together.

Example:

```
<com:automaticSplitAtInterchanges/>
```

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing
> **com:excludedFares** (Optional)

Please raise a JIRA ticket before using this element in your request.

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing excludedFares
> **com:ticketCode**

A three-character Fare Code to ignore when assessing split ticketing fares for the chosen journey.

Example:

```
<excludedFares>
  <ticketCode>SDS</ticketCode>
  <ticketCode>SDR</ticketCode>
</excludedFares>
```

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing **com:summatedFareFilters** (Optional)

A container element for one or more Fare Class and Fare Category filters.

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing summatedFareFilters **com:summatedFareFilter**

Defines a search criteria for finding Split Ticket fares.

Partners should include multiple `<summatedFareFilter>` elements to ensure that a variety of fare types (e.g., Standard vs. First Class, Advance vs. Walk-up) are returned in the response, especially when `maxSummatedFares` is low.

Since `maxSummatedFares` limits the results *per filter*, using specific filters guarantees that certain types of fares are included even if they are more expensive than others.

For example, if you search only with `fareClass="All"` and `maxSummatedFares="3"`, you might only receive Standard Class results if they are the cheapest option. To guarantee First Class results are also returned, you must add a second filter with `fareClass="First"`.

A fare is only returned once in the response, even if it is found by multiple fare filters.

The `summatedFareFilter` element has the following attributes:

Attribute	Description
`fareCategory`	The fare category to use to assess the journey for Split Ticket fares. Possible values are: - `All` (WalkOn and Advance) - `WalkOn` - `Advance`
`fareClass`	The fare class to use to assess the journey for Split Ticket fares. Possible values are: - `All` - `Standard` - `First`
`onlySingleFares`	Exclude return fares. False when omitted. Possible values are: - `True`

Attribute	Description
	- `False` When set to `False` (default), the response may include standard Return tickets that cover both the outbound and return journeys. In these cases, the corresponding `ticketableFare` will reference leg solutions from both journeys.

Example:

```
<summatedFareFilters>
❶ <summatedFareFilter fareCategory="All" fareClass="All"/>
❷ <summatedFareFilter fareCategory="All" fareClass="First"/>
</summatedFareFilters>
```

❶ Return the cheapest overall fares across all fare classes. ❷ Return the cheapest First Class fares.

> pointToPointShoppingRequest pointToPointShoppingQuery splitTicketing
> **com:includeFullSummatedFaresDetails** (Optional)

In addition to returning the pointToPointPrices for non-split fares, return a pointToPointPrice container for each summated fare, along with a reference to the summated fare the price is linked to.

Passing this parameter into a shopping request enables Partners to display fare details earlier in the sales funnel. Partners not wishing to pass this parameter into a shopping request are required to perform a validateBookingRecordRequest to display fare details.

As with the validation response, if a Ticketable Fare covers only part of a Travel Segment, SilverCore will also return the `subSegment` element, providing the origin and destination of that sub-segment and the departure and arrival times at those origin and destination calling points.

---

pointToPointShoppingResponse

In the pointToPointShoppingResponse, SilverCore will return the normal journey and fare results plus an additional container element, `summatedFares`, with the Split Fares and Leg

Solutions that those fares are valid for.

SilverCore will return a `summatedFare` element for each Split Fare that has a financial advantage over a Through Fare. In each instance of `summatedFare`, SilverCore also returns a reference to the Leg Solution(s) that the Split Fare is valid for.

In addition, if the request parameter `includeFullSummatedFaresDetails` was included in the request, the shopping response will include `pointToPointPrice` containers also for the Split Fares. As with the validation response, if a Ticketable Fare covers only part of a Travel Segment, SilverCore will also return the `subSegments` container, providing the origin and destination of each sub-segment and the departure and arrival times at those origin and destination calling points.

In the following example response, a single `summatedFareFilter` was used in the request:

```
<com:summatedFareFilters>
    <com:summatedFareFilter fareCategory="All" fareClass="All"/>
</com:summatedFareFilters>
```

Example pointToPointShoppingResponse (truncated):

```
<soap:Envelope
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <ns2:pointToPointShoppingResponse
            xmlns:ns2="http://railgds.net/ws/shopping"
            xmlns="http://railgds.net/ws/commontypes">
            <requestStatus systemId="QiET+0">
                <success>true</success>
            </requestStatus>
            <ns2:results>
                <ns2:legs>
                    <ns2:leg legID="L_1">
                        <originTravelPoint
type="STATION">GBQQM</originTravelPoint>
                        <destinationTravelPoint
type="STATION">GBSHF</destinationTravelPoint>
                        <legSolutions>
❶                        <legSolution legSolutionID="LS_1_0">
                                <numberOfConnections>0</numberOfConnections>
                                <serviceAlerts>
                                    <serviceAlert>
                                        <summary>This service may be busy due to
temporary short-notice changes to the timetable across the North</summary>
                                        <description/>
                                    </serviceAlert>
                                </serviceAlerts>
                                <travelSegments>
```

```
                                    <travelSegment sequence="0"
travelSegmentID="LS_1_0_TS_0" type="TRAIN">
                                        <originTravelPoint
type="STATION">GBQQM</originTravelPoint>
                                        <destinationTravelPoint
type="STATION">GBSHF</destinationTravelPoint>
                                        <departureDateTime>2022-10-
31T11:49:00</departureDateTime>
                                        <arrivalDateTime>2022-10-
31T13:06:00</arrivalDateTime>
                                        <designator>NT6155</designator>

<marketingCarrier>Northern</marketingCarrier>

<operatingCarrier>Northern</operatingCarrier>
                                        <supplierEquipmentType>Northern-
BLR</supplierEquipmentType>
                                        <duration>P0Y0M0DT1H17M0S</duration>
                                        <crossBorderInfo>false</crossBorderInfo>
                                        <equipmentType code="BLR">Branch-
Line/Regional</equipmentType>
                                        <carbonData unit="kgCO2">2.48</carbonData>
                                        <marketingServiceName>Northern service
from Manchester Piccadilly to Sheffield</marketingServiceName>
                                        <marketingInformation serviceCode=
"Northern"/>

<scheduleConfirmed>true</scheduleConfirmed>
                                    </travelSegment>
                                </travelSegments>
                                <overtakenJourney>false</overtakenJourney>
                                <duration>P0Y0M0DT1H17M0S</duration>
                                <passengerInformationRequired>
                                    <passengerInformation type="PASSENGER_NAME"
allPassengers="false"/>
                                </passengerInformationRequired>
                                <availableSeatPreferences>...
</availableSeatPreferences>
                            </legSolution>
                            <legSolution legSolutionID="LS_1_1">...</legSolution>
                            <legSolution legSolutionID="LS_1_2">...</legSolution>
                            <legSolution legSolutionID="LS_1_3">...</legSolution>
                            <legSolution legSolutionID="LS_1_4">...</legSolution>
                        </legSolutions>
                    </ns2:leg>
                </ns2:legs>
                <ns2:passengers>
                    <ns2:passenger passengerSpecID="PAX_SPEC_0">
                        <age>40</age>
                    </ns2:passenger>
                </ns2:passengers>
                <ns2:fareInformation>
                    <ns2:prices>...</ns2:prices>
```

```
                    <fareQualifiers>
                        <fareQualifier type="DISCOUNT_CARD"
fareQualifierID="GFQ_PRICE_P_1_55_0_0"    authorizationNumberRequired="false"
identifierRequired="false">
                            <program>UK_YOUNG_ADULT</program>
                        </fareQualifier>
                    </fareQualifiers>
                </ns2:fareInformation>
❷               <ns2:summatedFares>
                    <summatedFare summatedFareId="1">...</summatedFare>
                    <summatedFare summatedFareId="2">...</summatedFare>
                    <summatedFare summatedFareId="3">...</summatedFare>
                    <summatedFare summatedFareId="4">...</summatedFare>
                    <summatedFare summatedFareId="5">...</summatedFare>
                    <summatedFare summatedFareId="6">...</summatedFare>
❸                    <summatedFare summatedFareId="7">
                        <totalPrice currency="GBP">17.60</totalPrice>
❹                        <outwardFares>
                            <fare>CDS|NTH|00000|B5|0438|2826</fare>
                            <fare>SDS|NTH|01000||2826|6691</fare>
                        </outwardFares>
                        <outwardLegSolutions>
❺                           <legSolutionIDRef>LS_1_0</legSolutionIDRef>
                            <legSolutionIDRef>LS_1_3</legSolutionIDRef>
                        </outwardLegSolutions>
❻                        <summatedFareFilterIds>1</summatedFareFilterIds>
                    </summatedFare>
                </ns2:summatedFares>
❼               <ns2:summatedFareFilters>
                    <summatedFareFilter summatedFareFilterId="1"
fareCategory="All" fareClass="All" onlySingleFares="false"/>
                </ns2:summatedFareFilters>
            </ns2:results>
        </ns2:pointToPointShoppingResponse>
    </soap:Body>
</soap:Envelope>
```

❶ One of the outbound Leg Solutions returned in the response, `legSolutionID= "LS_1_0"`. ❷ The `summatedFares` container element with all the assessed Split Fares. ❸ One of the `summatedFare` elements returned for the chosen journey with an Id and price. ❹ The outbound Split Fare(s) found for the outward journey. ❺ The Ids of the Leg Solution that the Split Fare is valid for. Only one of these is passed into the `validateBookingRecordInformationRequest` and `createBookingRecordRequest`. ❻ If one or more `summatedFareFilter` elements have been passed into the request, the Id of the filter, as returned in `summatedFareFilters`, the summated fares correspond to. ❼ A container element summarizing the summated fare filters passed in the shopping request.

In this second shopping example, two `summatedFareFilters` were used in the request:

```
<com:summatedFareFilters>
    <com:summatedFareFilter fareCategory="All" fareClass="All"/>
    <com:summatedFareFilter fareCategory="All" fareClass="First"/>
</com:summatedFareFilters>
```

Example pointToPointShoppingResponse (truncated):

```
<ns2:summatedFares>
    <summatedFare summatedFareId="4">
        <totalPrice currency="GBP">158.70</totalPrice>
        <outwardFares>
            <fare>SDS|NTH|00000||0438|1243</fare>
            <fare>SOS|IWC|00000||1243|1072</fare>
        </outwardFares>
        <outwardLegSolutions>
            <legSolutionIDRef>LS_1_0</legSolutionIDRef>
        </outwardLegSolutions>
        <summatedFareFilterIds>1</summatedFareFilterIds>
    </summatedFare>
    <summatedFare summatedFareId="5">
        <totalPrice currency="GBP">241.90</totalPrice>
        <outwardFares>
            <fare>FDS|IXC|00000||0438|1268</fare>
            <fare>FOS|IWC|00000||1268|1072</fare>
        </outwardFares>
        <outwardLegSolutions>
            <legSolutionIDRef>LS_1_0</legSolutionIDRef>
        </outwardLegSolutions>
        <summatedFareFilterIds>2</summatedFareFilterIds>
    </summatedFare>
</ns2:summatedFares>
<ns2:summatedFareFilters>
    <summatedFareFilter summatedFareFilterId="1" fareCategory="All"
fareClass="All" onlySingleFares="false"/>
    <summatedFareFilter summatedFareFilterId="2" fareCategory="All"
fareClass="First" onlySingleFares="false"/>
</ns2:summatedFareFilters>
```

In this third example, the request parameter `includeFullSummatedFaresDetails` was set to `true` to return a `pointToPointPrice` container for each summated fare. A Fare Qualifier was also added:

```
<shop:fareQualifiers>
    <shop:fareQualifier type="DISCOUNT_CARD">
        <com:program>UK_NETWORK</com:program>
    </shop:fareQualifier>
</shop:fareQualifiers>
<shop:splitTicketing maxSplits="2" maxSummatedFares="3">
    <com:automaticSplitAtCallingStations/>
```

```
    <com:includeFullSummatedFaresDetails/>
</shop:splitTicketing>
```

Example pointToPointShoppingResponse (truncated):

```
<ns2:pointToPointPrice priceID="PRICE_P_2_85" summatedFareId="1">
    <totalPrice currency="GBP">33.25</totalPrice>
    <restrictiveFareClass>OFFPEAK_DAY_RETURN</restrictiveFareClass>
    <ticketableFares>
        <ticketableFare>
            <totalPrice currency="GBP">5.30</totalPrice>
            <passengerReferences>
                <passengerReference>
                    <passengerIDRef>PAX_SPEC_0</passengerIDRef>
                    <passengerTypeCode>A</passengerTypeCode>
                    <fareCodes>
                        <fareCode code="NSE-CDR-01000-STD-R">
                            <serviceClass>THIRD</serviceClass>
                            <travelSegmentIDRef>LS_2_6_TS_0</travelSegmentIDRef>
❶                            <subSegmentIDRef>LS_2_6_SS_0</subSegmentIDRef>
                            <cabinClass>Standard</cabinClass>
                            <rewardsEligible>NO</rewardsEligible>
                            <fareClass>OFFPEAK_DAY_RETURN</fareClass>
                            <fareDisplayName>Off-Peak Day Return</fareDisplayName>
                            <openReturn>NO</openReturn>
                            <reservable>NOT_POSSIBLE</reservable>
                            <fareExpirationDateTime>2024-12-
24T04:29:00Z</fareExpirationDateTime>
                            <seatsAvailable>0</seatsAvailable>
                            <fareApplicabilities>
                                <fareApplicability outbound="SEMI-OPEN"
return="SEMI-OPEN" type="SCHEDULE"/>
                            </fareApplicabilities>
                        </fareCode>
                    </fareCodes>
❷                <fareQualifierIDRef>GFQ_PRICE_P_2_85_0_0</fareQualifierIDRef>
                </passengerReference>
            </passengerReferences>
            ...
            <prices>...</prices>
            ...
            <rules>...</rules>
            ...
            </ticketableFare>
    </ticketableFares>
    <legReferences>
        <legSolutionIDRef>LS_1_0</legSolutionIDRef>
        <legSolutionIDRef>LS_2_6</legSolutionIDRef>
    </legReferences>
    <holdExpiration>2024-12-05T19:27:50Z</holdExpiration>
</ns2:pointToPointPrice>
```

❶ If Ticketable Fare covers only part of the Travel Segment, SilverCore indicates the sub-segment that the Ticketable Fare covers. ❷ The Fare Qualifier applied to the Ticketable Fare.

Response elements

Refer to the following descriptions for the details of each response element and attribute related to Split Ticketing in SilverCore.

> pointToPointShoppingResponse results legs leg legSolutions legSolution **subSegments**

If `includeFullSummatedFaresDetails` is included in your request and if a Ticketable Fare covers only part of a Travel Segment, SilverCore will also return the `subSegments` element, providing the origin and destination of that sub-segment and the departure and arrival times at those origin and destination calling points.

In the `ticketableFare` container, SilverCore will return `subSegmentIDRef` to indicate the sub-segment the Ticketable Fare covers.

> pointToPointShoppingResponse results legs leg legSolutions legSolution subSegments **subSegment**

The origin and destination of a sub-segment, the departure time at the `originTravelPoint` and the arrival time at the `destinationTravelPoint`.

Example:

```
<subSegment subSegmentID="LS_1_3_SS_0">
    <originTravelPoint type="STATION">GBQQN</originTravelPoint>
    <destinationTravelPoint type="STATION">GBBHX</destinationTravelPoint>
    <departureDateTime>2024-12-10T06:28:00</departureDateTime>
    <arrivalDateTime>2024-12-10T06:37:00</arrivalDateTime>
</subsegment>
```

> pointToPointShoppingResponse results **summatedFares**

A container element for one or more Split Fares evaluated and the associated Leg Solutions for the chosen journey.

> pointToPointShoppingResponse results summatedFares **summatedFare**

A Split Fare returned for the chosen journey. The `summatedFare` element has the following attributes:

Attribute	Description
`summatedFareId`	A unique Id for each Split Fare.

Example:

```
<summatedFare summatedFareId="1">
```

> pointToPointShoppingResponse results summatedFares summatedFare **totalPrice**

The price for this Split Fare. The `totalPrice` element has the following attribute:

Attribute	Description
`currency`	The currency in which the price is expressed (GBP, USD, CAD, EUR).

Example:

```
<totalPrice currency="GBP">189.00</totalPrice>
```

> pointToPointShoppingResponse results summatedFares summatedFare **outwardFares**

A container element for the Split Fare(s) returned for the outbound Leg Solutions for the chosen journey.

> pointToPointShoppingResponse results summatedFares summatedFare outwardFares **fare**

A Split Fare returned for the outbound Leg Solutions for the chosen journey.

Example:

```
<fare>SDS|IXC|00000||0438|1268</fare>
```

> pointToPointShoppingResponse results summatedFares summatedFare **returnFares**

For return journeys, a container element for the Split Fare(s) returned for one of the return Leg Solutions for the chosen journey.

> pointToPointShoppingResponse results summatedFares summatedFare returnFares **fare**

A Split Fare returned for one of the return Leg Solutions for the chosen journey.

Example:

```
<fare>V2L|IWC|00515|VR|1444|1243</fare>
```

> pointToPointShoppingResponse results summatedFares summatedFare **outwardLegSolutions**

A container element for the Ids of the outbound Leg Solutions that the Summated Fares are valid for.

> pointToPointShoppingResponse results summatedFares summatedFare outwardLegSolutions **legSolutionIDRef**

The Id of an outbound Leg Solution that the Split Fare is valid for.

Example:

```
<legSolutionIDRef>LS_1_1</legSolutionIDRef>
```

> pointToPointShoppingResponse results summatedFares summatedFare **returnLegSolutions**

A container element for the Ids of the return Leg Solutions that the Summated Fares are valid for.

> pointToPointShoppingResponse results summatedFares summatedFare returnLegSolutions **legSolutionIDRef**

The Id of a return Leg Solution that the Split Fare is valid for.

Example:

```
<legSolutionIDRef>LS_2_1</legSolutionIDRef>
```

> pointToPointShoppingResponse results summatedFares summatedFare **summatedFareFilterIds**

The Id of the summated fare filter that the summated fares correspond to, as returned in `summatedFareFilter`.

Example:

```
<summatedFareFilterIds>1</summatedFareFilterIds>
```

> pointToPointShoppingResponse results summatedFares **summatedFareFilters**

A container element for the summated fare filters passed into the shopping request.

> pointToPointShoppingResponse results summatedFares summatedFareFilters **summatedFareFilter**

A summated fare filter passed into the shopping request. SilverCore returns a unique Id along with the `fareCategory` and `fareClass`.

The `summatedFareFilter` has the following attributes:

Attribute	Description
`summatedFareFilterId`	A unique Id for each `summatedFareFilter` passed into the shopping request.
`fareCategory`	The fare category used to evaluate the journey for Split Ticket fares.
`fareClass`	The fare class used to evaluate the journey for Split Ticket fares.

Why do some Ticketable Fares reference both outbound and return journeys?

When `onlySingleFares` is set to `false` (default), SilverCore evaluates standard Return fares (e.g., Off-Peak Return) alongside combinations of Single fares.

If a single Return ticket covering the entire round trip is cheaper than separate Single tickets, SilverCore will select it. In these cases, the `ticketableFare` element will contain `legReference` elements for both the outbound and return Leg Solutions. This correctly indicates that one ticket covers segments from both journeys.

Validating a booking

Validating booking information allows you to preview the contents of the Booking Record before you create it.

validateBookingRecordInformationRequest

To validate your booking record for a split ticket fare, perform a validateBookingRecordInformationRequest and pass in the following data:

- The desired Leg Solution(s) from your `pointToPointShoppingResponse`.
- One of the desired and matching `summatedFare` containers from your `pointToPointShoppingResponse`.
- The same mandatory and optional split ticket elements you used for the `pointToPointShoppingRequest`.
- If you passed in a Fare Qualifier in your shopping request, echo back the data in the booking level `fareQualifier` container in your validate request.

One-way journeys

For one-way journeys, pass in the `summatedFare` container into your request with the desired `outwardFares` solution and one `legSolutionIdRef` from `outwardLegSolutions`.

The following example shows a `summatedFare` container for our one-way journey from Manchester Piccadilly to Sheffield. Note the reference back to a matching Leg Solution:

Example summatedFare container for a one-way journey:

```
<com:summatedFare summatedFareId="7">
    <com:totalPrice currency="GBP">17.60</com:totalPrice>
```

```
    <com:outwardFares>
        <com:fare>CDS|NTH|00000|B5|0438|2826</com:fare>
        <com:fare>SDS|NTH|01000||2826|6691</com:fare>
    </com:outwardFares>
    <com:outwardLegSolutions>
        <com:legSolutionIDRef>LS_1_0</com:legSolutionIDRef>
    </com:outwardLegSolutions>
    <com:summatedFareFilterIds>1</com:summatedFareFilterIds>
</com:summatedFare>
```

Return journeys

For return journeys, pass in the `summatedFare` container into your request with the desired `outwardFares` and `returnFares` solution, one `legSolutionIdRef` from `outwardLegSolutions`, and one `legSolutionIdRef` from `returnLegSolutions`.

Example summatedFare container for a return journey:

```
<com:summatedFare summatedFareId="1">
    <com:totalPrice currency="GBP">277.95</com:totalPrice>
    <com:outwardFares>
        <com:fare>SVR|IXC|00000|2V|0438|1268</com:fare>
        <com:fare>SVS|IWC|00000|2C|1268|1072</com:fare>
    </com:outwardFares>
    <com:returnFares>
        <com:fare>SOS|IWC|00000||1072|0418</com:fare>
        <com:fare>SDS|LBR|00000||0418|1268</com:fare>
        <com:fare>SVR|IXC|00000|2V|0438|1268</com:fare>
    </com:returnFares>
    <com:outwardLegSolutions>
        <com:legSolutionIDRef>LS_1_3</com:legSolutionIDRef>
    </com:outwardLegSolutions>
    <com:returnLegSolutions>
        <com:legSolutionIDRef>LS_2_9</com:legSolutionIDRef>
    </com:returnLegSolutions>
</com:summatedFare>
```

When you submit the `validateBookingRecordInformationRequest`, SilverCore performs a re-shop using the Leg Solution and Summated Fare data to ensure the tickets are still available and the pricing hasn't changed.

When working with Split Tickets, validating a booking also converts the Summated Fare into a `pointToPointPrice`, which is used in the create booking flow.

Example request for a one-way journey between Manchester Piccadilly and Sheffield:

```
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
```

```
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:validateBookingRecordInformationRequest version="2.0">
      <com:context>
        <com:distributorCode>DEMO</com:distributorCode>
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
      <book:passengers>...</book:passengers>
      <book:legSolutions>
❶     <com:legSolution legSolutionID="LS_1_0">
          <com:travelSegments>
            <com:travelSegment travelSegmentID="LS_1_0_TS_0" type="TRAIN">
              <com:originTravelPoint type="STATION">GBQQM</com:originTravelPoint>
              <com:destinationTravelPoint
type="STATION">GBSHF</com:destinationTravelPoint>
              <com:departureDateTime>2022-11-01T11:49:00</com:departureDateTime>
              <com:arrivalDateTime>2022-11-01T13:06:00</com:arrivalDateTime>
              <com:designator>NT6155</com:designator>
              <com:marketingCarrier>Northern</com:marketingCarrier>
              <com:equipmentType code="BLR">Branch-
Line/Regional</com:equipmentType>
            </com:travelSegment>
          </com:travelSegments>
        </com:legSolution>
      </book:legSolutions>
      <book:summatedFares>
❷     <com:summatedFare summatedFareId="7">
          <com:totalPrice currency="GBP">17.60</com:totalPrice>
          <com:outwardFares>
            <com:fare>CDS|NTH|00000|B5|0438|2826</com:fare>
            <com:fare>SDS|NTH|01000||2826|6691</com:fare>
          </com:outwardFares>
          <com:outwardLegSolutions>
❸         <com:legSolutionIDRef>LS_1_0</com:legSolutionIDRef>
          </com:outwardLegSolutions>
          <com:summatedFareFilterIds>1</com:summatedFareFilterIds>
        </com:summatedFare>
      </book:summatedFares>
❹     <book:splitTicketing maxSummatedFares="3" maxSplits="2">
        <com:automaticSplitAtCallingStations/>
        <com:excludedFares>
          <com:ticketCode>SDR</com:ticketCode>
        </com:excludedFares>
        <com:summatedFareFilters>
          <com:summatedFareFilter fareClass="All" fareCategory="All"/>
        </com:summatedFareFilters>
      </book:splitTicketing>
      <book:responseSpec>
   <book:returnReservationDetails>true</book:returnReservationDetails>
❺    <book:returnPricingDetails>true</book:returnPricingDetails>
```

```
      </book:responseSpec>
❻     <book:fareQualifiers>
         <book:fareQualifier type="DISCOUNT_CARD">
         <com:program>UK_YOUNG_ADULT</com:program>
         </book:fareQualifier>
      </book:fareQualifiers>
    </book:validateBookingRecordInformationRequest>
  </soap:Body>
</soap:Envelope>
```

❶ The `legSolution` container passed in from the shopping response. ❷ One of the matching fares from `summatedFares` passed in from the shopping response for the outward journey. ❸ A reference to the Leg Solution that the Summated Fare applies to, which must match the Id of the Leg Solution passed into the request. ❹ The mandatory and optional split ticket elements. ❺ Set `returnPricingDetails` to `true` to ensure your response message returns pricing details. ❻ The Fare Qualifier used in your `pointToPointShoppingRequest` should be echoed back in the booking level `fareQualifier` container in your validate request.

validateBookingRecordInformationResponse

In the `validateBookingRecordInformationResponse`, SilverCore will return the `pointToPointPrice` container for the chosen journey (for all the passengers on the booking), along with a `ticketableFare` element for each segment (or split) of the journey; Partners can use this information to display fare details.

If a Ticketable Fare covers only part of a Travel Segment, SilverCore will also return the `subSegment` element, providing the origin and destination of that sub-segment and the departure and arrival times at those origin and destination calling points.

In `ticketableFare`, SilverCore will return `subSegmentIDRef` to indicate the sub-segment the Ticketable Fare covers, as shown in the following example:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns2:validateBookingRecordInformationResponse
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
      <requestStatus systemId= "RUEsN/">
        <success>true</success>
      </requestStatus>
```

```
      <ns2:bookingRecordInformation>
        <ns2:orderInformationSet>
          <ns2:orderInformation>
            <ns2:passengers>...</ns2:passengers>
            <ns2:legSolutions>
              <legSolution legSolutionID="LS_1_0">
                <numberOfConnections>0</numberOfConnections>
                <travelSegments>
                  <travelSegment travelSegmentID="LS_1_0_TS_0" type="TRAIN">
                    <originTravelPoint type="STATION">GBQQM</originTravelPoint>
                    <destinationTravelPoint
type="STATION">GBQQU</destinationTravelPoint>
                    <departureDateTime>2023-07-07T06:55:00</departureDateTime>
                    <arrivalDateTime>2023-07-07T09:09:00</arrivalDateTime>
                    ...
                  </travelSegment>
                </travelSegments>
                <subSegments>
                  <subSegment subSegmentID="LS_1_0_SS_0">
                    <originTravelPoint type="STATION">GBQQM</originTravelPoint>
                    <destinationTravelPoint
type="STATION">GBCRE</destinationTravelPoint>
                    <departureDateTime>2023-07-07T06:55:00</departureDateTime>
                    <arrivalDateTime>2023-07-07T07:30:00</arrivalDateTime>
                  </subSegment>
                  <subSegment subSegmentID="LS_1_0_SS_1">
                    <originTravelPoint type="STATION">GBCRE</originTravelPoint>
                    <destinationTravelPoint
type="STATION">GBQQU</destinationTravelPoint>
                    <departureDateTime>2023-07-07T07:32:00</departureDateTime>
                    <arrivalDateTime>2023-07-07T09:09:00</arrivalDateTime>
                  </subSegment>
                </subSegments>
                ...
              </legSolution>
            </ns2:legSolutions>
            <ns2:prices>
              <ns2:pointToPointPrice priceID="PRICE_P_1_0">
                <totalPrice currency="GBP">50.60</totalPrice>
                <restrictiveFareClass>ADVANCE</restrictiveFareClass>
                <ticketableFares>
                  <ticketableFare>
                    <totalPrice currency="GBP">16.60</totalPrice>
                    <passengerReferences>
                      <passengerReference>
                        <passengerIDRef>PAX_SPEC_0</passengerIDRef>
                        <passengerTypeCode>A</passengerTypeCode>
                        <fareCodes>
                          <fareCode code="NTH-SDS-00000-STD-1">
                            <serviceClass>THIRD</serviceClass>
                            <travelSegmentIDRef>LS_1_0_TS_0</travelSegmentIDRef>
❶                          <subSegmentIDRef>LS_1_0_SS_0</subSegmentIDRef>
                            <cabinClass>Standard</cabinClass>
```

```
                            <rewardsEligible>NO</rewardsEligible>
                            <fareClass>ANYTIME_DAY</fareClass>
                            ...
                          </fareCode>
                        </fareCodes>
                      </passengerReference>
                    </passengerReferences>
                    <prices>
                      <price type="TICKET" currency="GBP">16.60</price>
                      <price type="RESERVATION" currency="GBP">0.00</price>
                    </prices>
                    ...
                    <ticketingOptionsAvailable TOD="true" ETK="true" PAH="false"
PRN="true" EML="false" DEPARTURE_STATION_TOD="true" SMS="false" XVD="true"
SCT="false" MVD="true"/>
                    ...
                  </ticketableFare>
                  <ticketableFare>...</ticketableFare>
                </ticketableFares>
                <legReferences>
                  <legSolutionIDRef>LS_1_0</legSolutionIDRef>
                </legReferences>
              </ns2:pointToPointPrice>
            </ns2:prices>
            ...
          </ns2:orderInformation>
        </ns2:orderInformationSet>
        ...
      </ns2:bookingRecordInformation>
    </ns2:validateBookingRecordInformationResponse>
  </soap:Body>
</soap:Envelope>
```

❶ `subSegmentIDRef` provides a reference to the sub-segment that the Ticketable Fare covers. This element must not be passed into your `createBookingRecordRequest` when copying the `pointToPointPrice` container.

Sub-segment examples

This section presents several examples demonstrating the circumstances in which SilverCore returns sub-segments.

Example 1 – Two Ticketable Fares covering one Travel Segment

Referring to our journey from Manchester Piccadilly to Sheffield, which has a single Travel Segment covered by two Ticketable Fares, SilverCore will return the two `subSegment elements` for the Travel Segment:

Example 2 – Two Ticketable Fares covering the first Travel Segment and one Ticketable Fare covering the second Travel Segment

If two Ticketable Fares cover the first Travel Segment and a single Ticketable Fare covers the second Travel Segment, SilverCore will return two `subSegment` elements for the first Travel Segment but not for the second:

**Example 3 – One Ticketable Fare covering part of the first Travel Segment and one Ticketable Fare covering the remainder of the first Travel Segment and the second Travel Segment **

If a Ticketable Fare only covers part of a Travel Segment and another Ticketable Fare covers the remainder of that Travel Segment and the whole of another Travel Segment, SilverCore will return the `subSegment` element for the first Travel Segment but not for the second:

**Example 4 – One Ticketable Fare covering the first Travel Segment and one Ticketable Fare covering the second Travel Segment **

If a Ticketable Fare covers a Travel Segment and another Ticketable Fare covers a second (or third, etc) Travel Segment, SilverCore will not return the `subSegment` element for either Travel Segment:

validateBookingRecordResponse for Example 1 (truncated):

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns2:validateBookingRecordInformationResponse>
      ...
      <ns2:bookingRecordInformation>
        <ns2:orderInformationSet>
          <ns2:orderInformation>
            <ns2:passengers>...</ns2:passengers>
            <ns2:legSolutions>
              <legSolution legSolutionID="LS_1_0">
                <numberOfConnections>0</numberOfConnections>
                 <travelSegments>
❶             <travelSegment travelSegmentID="LS_1_0_TS_0" type="TRAIN">
                  <originTravelPoint type="STATION">GBQQM</originTravelPoint>
                  <destinationTravelPoint
type="STATION">GBSHF</destinationTravelPoint>
                  <departureDateTime>2022-11-01T11:49:00</departureDateTime>
                  <arrivalDateTime>2022-11-01T13:06:00</arrivalDateTime>
                  .,.
                </travelSegment>
              </travelSegments>
❷            <subSegments>
              <subSegment subSegmentID="LS_1_0_SS_0">
                <originTravelPoint type="STATION">GBQQM</originTravelPoint>
```

```
                <destinationTravelPoint
type="STATION">GBHSG</destinationTravelPoint>
                <departureDateTime>2022-11-01T11:49:00</departureDateTime>
                <arrivalDateTime>2022-11-01T12:45:00</arrivalDateTime>
              </subSegment>
              <subSegment subSegmentID="LS_1_0_SS_1">
                <originTravelPoint type="STATION">GBHSG</originTravelPoint>
                <destinationTravelPoint
type="STATION">GBSHF</destinationTravelPoint>
                <departureDateTime>2022-11-01T12:45:00</departureDateTime>
                <arrivalDateTime>2022-11-01T13:06:00</arrivalDateTime>
              </subSegment>
            </subSegments>
            ...
          </legSolution>
        </ns2:legSolutions>
        <ns2:prices>
          <ns2:pointToPointPrice priceID="PRICE_P_1_0">
❸         <totalPrice currency="GBP">17.60</totalPrice>
          <restrictiveFareClass>OFFPEAK_DAY</restrictiveFareClass>
          <ticketableFares>
            <ticketableFare>
❹           <totalPrice currency="GBP">4.80</totalPrice>
            <passengerReferences>
              <passengerReference>
                <passengerIDRef>PAX_SPEC_0</passengerIDRef>
                <passengerTypeCode>A</passengerTypeCode>
                <fareCodes>
                <fareCode code="NTH-SDS-01000-STD-1">
                  <serviceClass>THIRD</serviceClass>
                  <travelSegmentIDRef>LS_1_0_TS_0</travelSegmentIDRef>
❺                 <subSegmentIDRef>LS_1_0_SS_1</subSegmentIDRef>
                  <cabinClass>Standard</cabinClass>
                  <rewardsEligible>NO</rewardsEligible>
                  <fareClass>ANYTIME_DAY</fareClass>
                  <fareDisplayName>Anytime Day Single</fareDisplayName>
                  ...
                </fareCode>
              </fareCodes>
            </passengerReference>
          </passengerReferences>
          <prices>
            <price type="TICKET" currency="GBP">4.80</price>
          </prices>
          <rules>...</rules>
          ...
      </ticketableFare>
      <ticketableFare>
❻     <totalPrice currency="GBP">12.80</totalPrice>
      <passengerReferences>
        <passengerReference>
          <passengerIDRef>PAX_SPEC_0</passengerIDRef>
          <passengerTypeCode>A</passengerTypeCode>
```

```
          <fareCodes>
            <fareCode code="NTH-CDS-00000-STD-1">
              <serviceClass>THIRD</serviceClass>
              <travelSegmentIDRef>LS_1_0_TS_0</travelSegmentIDRef>
              <subSegmentIDRef>LS_1_0_SS_0</subSegmentIDRef>
              <cabinClass>Standard</cabinClass>
              <rewardsEligible>NO</rewardsEligible>
              <fareClass>OFFPEAK_DAY</fareClass>
              <fareDisplayName>Off-Peak Day Single</fareDisplayName>
              ...
            </fareCode>
          </fareCodes>
        </passengerReference>
      </passengerReferences>
      <prices>
        <price type="TICKET" currency="GBP">12.80</price>
      </prices>
      ...
      <rules>...</rules>
     ...
    </ticketableFare>
  </ticketableFares>
  <legReferences>
    <legSolutionIDRef>LS_1_0</legSolutionIDRef>
  </legReferences>
</ns2:pointToPointPrice>
</ns2:prices>
...
</ns2:orderInformation>
</ns2:orderInformationSet>
...
</ns2:bookingRecordInformation>
</ns2:validateBookingRecordInformationResponse>
</soap:Body>
</soap:Envelope>
```

❶ The `travelSegment` container details the chosen journey from Manchester Piccadilly to Sheffield. The `travelSegmentId` indicates the travel segment which the fare applies to. ❷ If the fare covers only a portion of the Travel Segment, the response will include a `subSegment` container element, which provides details of the part of the Travel Segment that the fare covers. ❸ The total price for the journey. ❹ The price for the leg from GBHSG to GBSHF. ❺ A reference to the sub-segment that the Ticketable Fare applies to. This element must not be passed into your `createBookingRecordRequest` when copying the `pointToPointPrice` container. ❻ The price for the leg from GBMPV to GBHSG.

Creating a booking

Creating bookings for Split Ticket fares is done through the `createBookingRecordRequest` and mirrors the flow for booking Point-To-Point fares.

createBookingRecordRequest

To create a booking for a Split Ticket fare, perform a createBookingRecordRequest and pass in the following data:

- The desired Leg Solution(s) from your `pointToPointShoppingResponse`.
- The matching `summatedFare` container from the `validateBookingRecordInformationRequest`.
- The same mandatory and optional split ticket data you used for the `pointToPointShoppingRequest`.
- The `pointToPointPrice` from your `validateBookingRecordInformationResponse` to confirm that the stated prices are still available and haven't changed:
  - Remove the subSegmentIDRef element from your `createBookingRecordRequest`.

One-way journeys

For one-way journeys, pass in the `summatedFare` container into your request with the desired `outwardFares` solution and one `legSolutionIdRef` from `outwardLegSolutions`.

Return journeys

For return journeys, pass in the `summatedFare` container into your request with the desired `outwardFares` and `returnFares` solution, one `legSolutionIdRef` from `outwardLegSolutions`, and one `legSolutionIdRef` from `returnLegSolutions`.

When you submit the `createBookingRecordRequest` message, SilverCore performs a re-shop using the Leg Solution, `pointToPointPrice`, and `summatedFare` data to ensure the tickets are still available and the pricing hasn't changed.

Example request for a one-way journey between Manchester Piccadilly and Sheffield:

```
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:createBookingRecordRequest version="2.0">
      <com:context>
        <com:distributorCode>DEMO</com:distributorCode>
```

```
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
      <book:passengers>...<book:legSolutions>
❶    <com:legSolution legSolutionID="LS_1_0">
          <com:travelSegments>
            <com:travelSegment travelSegmentID="LS_1_0_TS_0" type="TRAIN">
              <com:originTravelPoint type="STATION">GBQQM</com:originTravelPoint>
              <com:destinationTravelPoint
type="STATION">GBSHF</com:destinationTravelPoint>
              <com:departureDateTime>2022-11-01T11:49:00</com:departureDateTime>
              <com:arrivalDateTime>2022-11-01T13:06:00</com:arrivalDateTime>
              <com:designator>NT6155</com:designator>
              <com:marketingCarrier>Northern</com:marketingCarrier>
              <com:equipmentType code="BLR">Branch-
Line/Regional</com:equipmentType>
            </com:travelSegment>
          </com:travelSegments>
        </com:legSolution>
      </book:legSolutions>
❷ <book:prices>
        <book:pointToPointPrice priceID="PRICE_P_1_0">
          <com:totalPrice currency="GBP">17.60</com:totalPrice>
          <com:ticketableFares>
            <com:ticketableFare>
              <com:totalPrice currency="GBP">4.80</com:totalPrice>
              <com:passengerReferences>
                <com:passengerReference>
                  <com:passengerIDRef>PAX_SPEC_0</com:passengerIDRef>
                  <com:passengerTypeCode>A</com:passengerTypeCode>
                  <com:fareCodes>
                    <com:fareCode code="NTH-SDS-01000-STD-1">
                      <com:serviceClass>THIRD</com:serviceClass>
                      <com:travelSegmentIDRef>LS_1_0_TS_0</com:travelSegmentIDRef>
                      <com:cabinClass>Standard</com:cabinClass>
                    </com:fareCode>
                  </com:fareCodes>
                </com:passengerReference>
              </com:passengerReferences>
              <com:prices>
                <com:price currency="GBP" type="TICKET">4.80</com:price>
              </com:prices>
              <com:fareOrigin type="STATION">GBHSG</com:fareOrigin>
              <com:fareDestination type="STATION">GBSHF</com:fareDestination>
            </com:ticketableFare>
            <com:ticketableFare>
              <com:totalPrice currency="GBP">12.80</com:totalPrice>
              <com:passengerReferences>
                <com:passengerReference>
                  <com:passengerIDRef>PAX_SPEC_0</com:passengerIDRef>
                  <com:passengerTypeCode>A</com:passengerTypeCode>
                  <com:fareCodes>
                    <com:fareCode code="NTH-CDS-00000-STD-1">
```

```
                      <com:serviceClass>THIRD</com:serviceClass>
                      <com:travelSegmentIDRef>LS_1_0_TS_0</com:travelSegmentIDRef>
                      <com:cabinClass>Standard</com:cabinClass>
                    </com:fareCode>
                  </com:fareCodes>
                </com:passengerReference>
              </com:passengerReferences>
              <com:prices>
                <com:price currency="GBP" type="TICKET">12.80</com:price>
              </com:prices>
              <com:fareOrigin type="STATION">GBMPV</com:fareOrigin>
              <com:fareDestination type="STATION">GBHSG</com:fareDestination>
            </com:ticketableFare>
          </com:ticketableFares>
          <com:legReferences>
            <com:legSolutionIDRef>LS_1_0</com:legSolutionIDRef>
          </com:legReferences>
        </book:pointToPointPrice>
      </book:prices>
❸   <book:summatedFares>
        <com:summatedFare summatedFareId="7">
          <com:totalPrice currency="GBP">17.60</com:totalPrice>
          <com:outwardFares>
            <com:fare>CDS|NTH|00000|B5|0438|2826</com:fare>
            <com:fare>SDS|NTH|01000||2826|6691</com:fare>
          </com:outwardFares>
          <com:outwardLegSolutions>
            <com:legSolutionIDRef>LS_1_0</com:legSolutionIDRef>
          </com:outwardLegSolutions>
          <com:summatedFareFilterIds>1</com:summatedFareFilterIds>
        </com:summatedFare>
      </book:summatedFares>
❹  <book:splitTicketing maxSummatedFares="3" maxSplits="2">
        <com:automaticSplitAtCallingStations/>
        <com:excludedFares>
          <com:ticketCode>SDR</com:ticketCode>
        </com:excludedFares>
        <com:summatedFareFilters>
          <com:summatedFareFilter fareClass="All" fareCategory="All"/>
        </com:summatedFareFilters>
      </book:splitTicketing>
      <book:responseSpec>
        <book:returnReservationDetails>true</book:returnReservationDetails>
        <book:returnPricingDetails>true</book:returnPricingDetails>
      </book:responseSpec>
    </book:createBookingRecordRequest>
  </soap:Body>
</soap:Envelope>
```

❶ The chosen Leg Solution from your `pointToPointShoppingReponse`. ❷ The prices container from your `pointToPointShoppingResponse` to verify the availability of the stated ticket price. Note that the `subSegmentIdRef` element has been removed. ❸ The

`summatedFare` from your `pointTopointShoppingResponse` that matches the chosen Leg Solution. ❹ The mandatory and optional split ticket elements.

createBookingRecordResponse

The createBookingRecordResponse shows the success/fail status for the booking, assigns a unique Booking Record locator Id, and indicates the hold date/time after which the booking will be canceled if not paid and confirmed.

If `<book:returnReservationDetails>true</book:returnReservationDetails>` is passed in the request, the response message will echo back a complete Booking Record containing passenger information, Travel Segments, and Ticketable Fares along with full payment status and financials.

As with the `validateBookingRecordResponse`, if a Ticketable Fare covers only part of a Travel Segment, SilverCore will also return the `subSegment` element, providing the origin and destination of that sub-segment, the departure and arrival times at those origin and destination calling points, and a reference back to the Ticketable Fare.

The booking response also returns a `combinedTicketableFareSets` container element to indicate within the order which of the Ticketable Fares were generated from a Summated Fare.

The container is returned in the create booking response and any subsequent booking retrievals. SilverCore returns a set for both outward and return legs of the journey (if a return journey) but only if the Ticketable Fares for that leg were generated from Summated Fares.

Example combinedTicketableFareSets container for a return journey:

```
<combinedTicketableFareSets>
    <combinedTicketableFareSet combinedTicketableFareSetId="CTF-1">
        <combinedTicketableFareLocators>
            <combinedTicketableFareLocator>TF-XHL000004-CHR000001-
0</combinedTicketableFareLocator>
            <combinedTicketableFareLocator>TF-XHL000004-CHR000001-
1</combinedTicketableFareLocator>
        </combinedTicketableFareLocators>
    </combinedTicketableFareSet>
    <combinedTicketableFareSet combinedTicketableFareSetId="CTF-2">
        <combinedTicketableFareLocators>
            <combinedTicketableFareLocator>TF-XHL000004-CHR000001-
2</combinedTicketableFareLocator>
            <combinedTicketableFareLocator>TF-XHL000004-CHR000001-
3</combinedTicketableFareLocator>
        </combinedTicketableFareLocators>
```

```
    </combinedTicketableFareSet>
</combinedTicketableFareSets>
```

Example createBookingRecordResponse (truncated):

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns2:createBookingRecordResponse
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
      <requestStatus systemId="BThwlt">
        <success>true</success>
      </requestStatus>
      <ns2:recordLocator>B-DEMO-BAN0001X5</ns2:recordLocator>
      <ns2:lastHoldDateTime>2022-10-23T12:23:44Z</ns2:lastHoldDateTime>
❶     <ns2:bookingRecord recordLocator="B-DEMO-BAN0001X5" status="DEBIT">
        <updateable>
          <passengerIdentity>true</passengerIdentity>
          <passengerAddress>true</passengerAddress>
          <passengerContactInfo>true</passengerContactInfo>
          <loyaltyCard>true</loyaltyCard>
          <travelDocument>true</travelDocument>
        </updateable>
        <bookingDate>2022-10-23Z</bookingDate>
        <departureDate>2022-11-01</departureDate>
        <numberOfOrders>1</numberOfOrders>
❷       <revenueTotal currency="GBP">17.60</revenueTotal>
        <receiptsTotal currency="GBP">0.00</receiptsTotal>
        <passengers>...</passengers>
        <orders>
          <order orderID="O-DEMO-BAN0001X5-FXH000001" status="BOOKED">
            <dateBooked>2022-10-23Z</dateBooked>
            <serviceAlerts>
              <serviceAlert>
                <summary>This service may be busy due to temporary short notice
changes to the timetable across the North</summary>
                <description/>
              </serviceAlert>
            </serviceAlerts>
            <holdExpiration>2022-10-23T12:23:44Z</holdExpiration>
            <refundEligibility eligible="true" cancelRequiredForRefund="true">
              <penalty currency="GBP">0.00</penalty>
            </refundEligibility>
            <cancellationSummary isCancellable="true"
isPartiallyCancellable="true">
              <cancellationOptions>
                <cancellationOption>
```

```
                <refundTarget type="FORM_OF_PAYMENT"/>
                <penalty>
                  <total currency="GBP">0.00</total>
                </penalty>
              </cancellationOption>
            </cancellationOptions>
            <priceReversals>
              <total currency="GBP">17.60</total>
              <breakdown>
                <component type="Product_Sale">
                  <amount currency="GBP">17.60</amount>
                </component>
              </breakdown>
            </priceReversals>
          </cancellationSummary>
          <availableTicketLanguages>
            <language>en</language>
          </availableTicketLanguages>
          <updateable>
            <ticketOption>true</ticketOption>
            <supplierNote>true</supplierNote>
          </updateable>
          <travelSegments>...</travelSegments>
❸         <subSegments>
            <subSegment subSegmentID="SS-BAN0001X5-FXH000001-1">
              <originTravelPoint type="STATION">GBQQM</originTravelPoint>
              <destinationTravelPoint
type="STATION">GBHSG</destinationTravelPoint>
              <departureDateTime>2022-11-01T11:49:00</departureDateTime>
              <arrivalDateTime>2022-11-01T12:45:00</arrivalDateTime>
            </subSegment>
            <subSegment subSegmentID="SS-BAN0001X5-FXH000001-2">
              <originTravelPoint type="STATION">GBHSG</originTravelPoint>
              <destinationTravelPoint
type="STATION">GBSHF</destinationTravelPoint>
              <departureDateTime>2022-11-01T12:45:00</departureDateTime>
              <arrivalDateTime>2022-11-01T13:06:00</arrivalDateTime>
            </subSegment>
          </subSegments>
          <ticketableFares>...</ticketableFares>
          <ticketingOptions>...</ticketingOptions>
          <supplyChannelPassengers>
            <supplyChannelPassengerReference>
              <passengerIDRef>P-BAN0001X5-0</passengerIDRef>
              <nameFirst>Neveah</nameFirst>
              <nameLast>Molina</nameLast>
            </supplyChannelPassengerReference>
          </supplyChannelPassengers>

<confirmationInformationRequired>false</confirmationInformationRequired>

<paymentCardRequiredForConfirmationInformation>false</paymentCardRequiredForConfir
mationInformation>
```

```
            <confirmationOptions>...</confirmationOptions>
            <commercialAgentName>SilverRail Technologies UK Ltd -
3258</commercialAgentName>
❹           <combinedTicketableFareSets>
              <combinedTicketableFareSet combinedTicketableFareSetId="CTF_0">
                <combinedTicketableFareLocators>
                  <combinedTicketableFareLocator>TF-DXT0004OY-VJP000001-
0</combinedTicketableFareLocator>
                  <combinedTicketableFareLocator>TF-DXT0004OY-VJP000001-
1</combinedTicketableFareLocator>
                </combinedTicketableFareLocators>
               </combinedTicketableFareSet>
             </combinedTicketableFareSets>
          </order>
        </orders>
❺       <paymentRequirements>
         <paymentDue>2022-10-23T12:23:44Z</paymentDue>
          <paymentInformation>ORDER_TOTAL</paymentInformation>
          <nextDepositDue>2022-10-23T12:23:44Z</nextDepositDue>
          <minimumDepositAmount currency="GBP">17.60</minimumDepositAmount>
          <acceptableFormsOfPayment>
            <formOfPayment type="OA">EP</formOfPayment>
            <formOfPayment type="OA">NONE</formOfPayment>
          </acceptableFormsOfPayment>
          <refundIsProcessedOnCancel>false</refundIsProcessedOnCancel>
          <confirmationUponAddPayment>false</confirmationUponAddPayment>
        </paymentRequirements>
        <serviceFeeAllowed>true</serviceFeeAllowed>
        <creditCardSurchargeFeeApplicable>false</creditCardSurchargeFeeApplicable>
        <financials>
          <prices>
            <price priceID="FIN_PRICE_9587" type="Product_Sale"
priceDescription="Product Sale:O-DEMO-BAN0001X5-FXH000001">
              <orderIDRef>O-DEMO-BAN0001X5-FXH000001</orderIDRef>
              <amount currency="GBP">17.60</amount>
              <postingDate>2022-10-23Z</postingDate>
              <postingTime>10:38:44Z</postingTime>
            </price>
          </prices>
        </financials>
      </ns2:bookingRecord>
    </ns2:createBookingRecordResponse>
  </soap:Body>
</soap:Envelope>
```

❶ The unique Booking Record locator Id. ❷ The total price for the journey. Note the difference between `revenueTotal` and `receiptsTotal`. When a payment is made for the booking, the financials will be balanced. ❸ If the fare covers only a portion of the travel segment, the response will include a `subSegment` container element, which provides details of the part of the travel segment that the fare covers. ❹ The `combinedTicketableFareSets` container indicates which Ticketable Fares were generated from a Summated Fare and

which Ticketable Fares must be canceled together when performing a partial cancel. ❺ In the `paymentDue` element in the financials container, the response indicates the hold date/time after which the booking will be canceled if not paid and confirmed.

Exchanging Split Tickets

Split Ticket exchanges are supported for one-way journeys that contain Advance fares (either exclusively or mixed with Walk-up fares). The exchange process for Split Tickets mirrors the standard exchange flow but involves handling multiple tickets resulting from the split solution.

exchangeSearchRequest

To search for exchange options for a Split Ticket booking, perform an `exchangeSearchRequest` and include the `splitTicketing` element in the `exchangeShoppingQuery`. This explicitly requests split ticket solutions for the replacement journey.

Example exchangeSearchRequest:

```
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Header/>
    <soap:Body>
        <book:exchangeSearchRequest version="2.0">
            <com:context>
                <com:distributorCode>DEMO</com:distributorCode>
                <com:pointOfSaleCode>GB</com:pointOfSaleCode>
                <com:channelCode>IOS</com:channelCode>
            </com:context>
            <book:recordLocator>B-DEMO-HBA00004G</book:recordLocator>
            <book:orderLocator>O-DEMO-HBA00004G-UKO000001</book:orderLocator>
            <book:exchangeSets>
                <book:exchangeSet exchangeSetID="EXCH-1">
❶                   <com:ticketableFareLocators>
                        <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
0</com:ticketableFareLocator>
                        <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
1</com:ticketableFareLocator>
                        <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
2</com:ticketableFareLocator>
                    </com:ticketableFareLocators>
                    <book:exchangeShoppingQuery>
                        <book:travelPointPairs>
                            <book:travelPointPair>
```

```
                                <com:originTravelPoint
type="STATION">GBQQM</com:originTravelPoint>
                                <com:destinationTravelPoint
type="STATION">GBQQU</com:destinationTravelPoint>
                                <com:departureDateTimeWindow>
                                    <com:date>2026-01-13</com:date>
                                </com:departureDateTimeWindow>
                            </book:travelPointPair>
                        </book:travelPointPairs>
❷                        <book:splitTicketing maxSplits="2" maxSummatedFares="3">
                            <com:automaticSplitAtCallingStations />
                            <com:summatedFareFilters>
                                <com:summatedFareFilter fareCategory="All"
fareClass="All" />
                            </com:summatedFareFilters>
                            <com:includeFullSummatedFaresDetails />
                        </book:splitTicketing>
                    </book:exchangeShoppingQuery>
                </book:exchangeSet>
            </book:exchangeSets>
        </book:exchangeSearchRequest>
    </soap:Body>
</soap:Envelope>
```

❶ Specify the `ticketableFareLocator`s for ALL tickets in the original order that you wish to exchange. ❷ The `splitTicketing` element is required in the `exchangeShoppingQuery` to return split ticketing solutions.

exchangeSearchResponse

When performing an `exchangeSearchRequest` for a one-way Split Ticketing booking: - The exchange search results contain only split fares (no through fares). - The fares are split at the same locations as the original booking. - The exchange search results include sub-segments and split ticketing parameters, consistent with the `pointToPointShoppingResponse`. - The response returns `pointToPointPrice` elements for each split ticketing solution. Unlike the standard shop response which returns `summatedFare` elements, the exchange search response provides the expanded `pointToPointPrice` structure directly, containing multiple `ticketableFare` elements (one for each leg of the split).

retrieveExchangeSummaryRequest

When retrieving the summary for a split ticket exchange, you must pass the `legSolution` and the `pointToPointPrice` corresponding to the chosen split solution.

Example retrieveExchangeSummaryRequest:

```
<book:retrieveExchangeSummaryRequest version="2.0">
    <com:context>
        ...
    </com:context>
    <book:exchangeSets>
        <book:exchangeSet exchangeSetID="EXCH-1">
❶           <com:ticketableFareLocators>
                <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
0</com:ticketableFareLocator>
                <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
1</com:ticketableFareLocator>
                <com:ticketableFareLocator>TF-HBA00004G-UKO000001-
2</com:ticketableFareLocator>
            </com:ticketableFareLocators>
            <book:legSolutions>...</book:legSolutions>
            <book:prices>
❷               <book:pointToPointPrice priceID="PRICE_P_1_0">
                    <com:totalPrice currency="GBP">119.80</com:totalPrice>
❸                   <com:ticketableFares>
                        <com:ticketableFare>
                            <com:totalPrice currency="GBP">59.90</com:totalPrice>
                            ...
                        </com:ticketableFare>
                        <com:ticketableFare>
                            <com:totalPrice currency="GBP">59.90</com:totalPrice>
                            ...
                        </com:ticketableFare>
                    </com:ticketableFares>
                </book:pointToPointPrice>
            </book:prices>
        </book:exchangeSet>
    </book:exchangeSets>
</book:retrieveExchangeSummaryRequest>
```

❶ The original Ticketable Fares being exchanged. ❷ The `pointToPointPrice` returned in the `exchangeSearchResponse` that represents the chosen split solution. ❸ The expanded `ticketableFares` representing the multiple segments of the split journey.

processExchangeRequest

The `processExchangeRequest` follows the standard format, accepting the same `legSolution` and `prices` structure used in the summary request.

Canceling bookings with Split Tickets

Canceling bookings with Split Ticket fares in SilverCore follows the process followed for Point-To-Point fares, with some important exceptions:

- To prevent fraud, canceling individual Split Tickets on an order is not supported. Either the entire order must be canceled (full cancel), or all the Ticketable Fares for a given Leg Solution must be canceled together (partial cancel).
- Split Tickets with Advance fares cannot be canceled in SilverCore.
- Orders with a mixture of Advance and Anytime fares cannot be canceled in SilverCore.

Full cancel

Step 1 – Find out if an order can be fully canceled

retrieveBookingRecordRequest

Before proceeding with the full order cancellation, Partners must first determine whether the order can be fully canceled. To do this, perform a retrieveBookingRecordRequest passing in the following data:

- `recordLocator`

retrieveBookingRecordResponse

In the retrieveBookingRecordResponse, SilverCore returns a `cancellationSummary` container indicating whether a cancellation is allowed for the order and, if so, what penalties apply.

An order is eligible for full cancellation if `isCancellable="true"` is returned in the `cancellationSummary` container element:

```
<cancellationSummary isCancellable="true" isPartiallyCancellable="false">
```

Example retrieveBookingRecordResponse:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
      <ns2:retrieveBookingRecordResponse
conversationToken="SplitTicketing_Shop_Validate_Book_Demo"
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
         <requestStatus systemId="XGjTZO">
            <success>true</success>
         </requestStatus>
         <ns2:bookingRecord recordLocator="B-DEMO-EPE000291" status="DEBIT">
```

```
            <updateable>
               <passengerIdentity>true</passengerIdentity>
               <passengerAddress>true</passengerAddress>
               <passengerContactInfo>true</passengerContactInfo>
               <loyaltyCard>true</loyaltyCard>
               <travelDocument>true</travelDocument>
            </updateable>
            <bookingDate>2023-06-29Z</bookingDate>
            <departureDate>2023-07-04</departureDate>
            <numberOfOrders>1</numberOfOrders>
            <revenueTotal currency="GBP">44.20</revenueTotal>
            <receiptsTotal currency="GBP">0.00</receiptsTotal>
            <passengers>...</passengers>
            <orders>
               <order orderID="O-DEMO-EPE000291-EXF000001" status="BOOKED">
                  <dateBooked>2023-06-29Z</dateBooked>
                  <holdExpiration>2023-06-29T13:20:44Z</holdExpiration>
                  <refundEligibility eligible="true"
cancelRequiredForRefund="true">
                     <penalty currency="GBP">0.00</penalty>
                  </refundEligibility>
❶                 <cancellationSummary isCancellable="true"
isPartiallyCancellable="true">
                     <cancellationOptions>
                        <cancellationOption>
                           <refundTarget type="FORM_OF_PAYMENT"/>
                           <penalty>
                              <total currency="GBP">0.00</total>
                           </penalty>
                        </cancellationOption>
                     </cancellationOptions>
                     <priceReversals>
                        <total currency="GBP">44.20</total>
                        <breakdown>
                           <component type="Product_Sale">
                              <amount currency="GBP">22.10</amount>
                           </component>
                           <component type="Product_Sale">
                              <amount currency="GBP">22.10</amount>
                           </component>
                        </breakdown>
                     </priceReversals>
                  </cancellationSummary>
                  ...
                  <combinedTicketableFareSets>
                     <combinedTicketableFareSet
combinedTicketableFareSetId="CTF_0">
                        <combinedTicketableFareLocators>
                           <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
0</combinedTicketableFareLocator>
                           <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
1</combinedTicketableFareLocator>
                        </combinedTicketableFareLocators>
```

```
                    </combinedTicketableFareSet>
                    <combinedTicketableFareSet
combinedTicketableFareSetId="CTF_1">
                       <combinedTicketableFareLocators>
                          <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
2</combinedTicketableFareLocator>
                          <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
3</combinedTicketableFareLocator>
                       </combinedTicketableFareLocators>
                    </combinedTicketableFareSet>
                 </combinedTicketableFareSets>
              </order>
           </orders>
           ...
        </ns2:bookingRecord>
     </ns2:retrieveBookingRecordResponse>
</soap:Body>
```

❶ The order can be fully canceled.

Step 2 – Fully cancel the order

cancelBookingRecordRequest

To fully cancel a booking, perform a cancelBookingRecordRequest and pass in the following data:

- `recordLocator` from your `retrieveBookingRecordResponse`.
- `orderLocator` from your `retrieveBookingRecordResponse`.
- `expectedCancellationFee`, which must match the penalty total returned in the `retrieveBookingRecordResponse`.

> **Warning**
>
> To perform a full order cancellation, do not pass in all the Ticketable Fares on the order.

Example cancelBookingRecordRequest:

```
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:cancelBookingRecordRequest version="2.0">
      <com:context>
```

```
        <com:distributorCode>DEMO</com:distributorCode>
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
❶     <book:recordLocator>B-DEMO-BAN0001X5</book:recordLocator>
❷     <book:orderLocator>O-DEMO-BAN0001X5-KCU000001</book:orderLocator>
❸     <book:expectedCancellationFee
currency="GBP">0.00</book:expectedCancellationFee>
      <book:responseSpec>
        <book:returnReservationDetails>true</book:returnReservationDetails>
      </book:responseSpec>
    </book:cancelBookingRecordRequest>
  </soap:Body>
</soap:Envelope>
```

❶ The record locator Id for the booking to cancel. ❷ The order locator Id for the order on the booking to cancel. ❸ No fees are expected to be charged for this cancellation.

cancelBookingRecordResponse

The resulting cancelBookingRecordResponse message shows the success/fail status for the cancellation and optionally echoes back a complete Booking Record containing passenger information, Travel Segments, and Ticketable Fares, along with full payment status and financials. It also includes information about the status of any refunds.

Example cancelBookingRecordResponse (truncated):

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns2:cancelBookingRecordResponse xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
      <requestStatus systemId="9C6JGk">
        <success>true</success>
      </requestStatus>
❶     <ns2:bookingRecord recordLocator="B-DEMO-OYF000010" status="CREDIT">
        <bookingDate>2022-10-25Z</bookingDate>
        <departureDate>2022-11-03</departureDate>
        <numberOfOrders>1</numberOfOrders>
❷       <revenueTotal currency="GBP">0.00</revenueTotal>
        <receiptsTotal currency="GBP">17.60</receiptsTotal>
        <passengers>...</passengers>
        <orders>
❸          <order orderID="O-DEMO-OYF000010-TXX000001" status="CANCELED">
            <dateBooked>2022-10-25Z</dateBooked>
            <dateCancelled>2022-10-25T15:06:18Z</dateCancelled>
            <holdExpiration>2022-10-25T16:49:56Z</holdExpiration>
            <fulfillmentInformation>...</fulfillmentInformation>
            <refundEligibility eligible= "false"/>
❹           <cancellationSummary isCancellable="false"
isPartiallyCancellable="false"/>
```

```
            <cancellationReason>FULL_TICKET_NOT_REQD</cancellationReason>
            <availableTicketLanguages>
              <language>en</language>
            </availableTicketLanguages>
            <isInventoryCanceled>true</isInventoryCanceled>
            <travelSegments>...</travelSegments>
            <subSegments>...</subSegments>
            <ticketableFares>...</ticketableFares>
            <ticketingOptions>...</ticketingOptions>
            <supplyChannelPassengers>
              <supplyChannelPassengerReference>
                <passengerIDRef>P-OYF000010-0</passengerIDRef>
                <nameFirst>Minerva</nameFirst>
                <nameLast>Richards</nameLast>
              </supplyChannelPassengerReference>
            </supplyChannelPassengers>

<confirmationInformationRequired>false</confirmationInformationRequired>

<paymentCardRequiredForConfirmationInformation>false</paymentCardRequiredForConfir
mationInformation>
            <commercialAgentName>1427</commercialAgentName>
          </order>
        </orders>
        <paymentRequirements>...</paymentRequirements>
        <serviceFeeAllowed>true</serviceFeeAllowed>
        <creditCardSurchargeFeeApplicable>false</creditCardSurchargeFeeApplicable>
        <financials>
          <prices>
            <price priceID="FIN_PRICE_105" type="Product_Sale"
priceDescription="Product Sale:O-DEMO-OYF000010-TXX000001">
              <orderIDRef>O-DEMO-OYF000010-TXX000001</orderIDRef>
              <amount currency="GBP">17.60</amount>
              <postingDate>2022-10-25Z</postingDate>
              <postingTime>15:04:56Z</postingTime>
            </price>
            <price priceID="FIN_PRICE_106" type="Ticket_Delivery_Fee_Assessment"
priceDescription="Ticket Delivery Fee:O-DEMO-OYF000010-TXX000001">
              <orderIDRef>O-DEMO-OYF000010-TXX000001</orderIDRef>
              <amount currency="GBP">0.00</amount>
              <postingDate>2022-10-25Z</postingDate>
              <postingTime>15:05:55Z</postingTime>
            </price>
            <price priceID="FIN_PRICE_107" type="Product_Refund"
priceDescription="Product Sale:O-DEMO-OYF000010-TXX000001">
              <orderIDRef>O-DEMO-OYF000010-TXX000001</orderIDRef>
              <amount currency="GBP">-17.60</amount>
              <postingDate>2022-10-25Z</postingDate>
              <postingTime>15:06:18Z</postingTime>
            </price>
            <price priceID="FIN_PRICE_108" type="Cancelation_Penalty_Assessment"
priceDescription="Cancellation Penalty:O-DEMO-OYF000010-TXX000001">
              <orderIDRef>O-DEMO-OYF000010-TXX000001</orderIDRef>
```

```
                <amount currency="GBP">0.00</amount>
                <postingDate>2022-10-25Z</postingDate>
                <postingTime>15:06:18Z</postingTime>
              </price>
            </prices>
            <payments>...</payments>
          </financials>
        </ns2:bookingRecord>
      </ns2:cancelBookingRecordResponse>
    </soap:Body>
  </soap:Envelope>
```

❶ This cancellation will result in a refund being due to the customer. ❷ The cancellation reverses the financials for the order. ❸ The order status is updated to `CANCELLED`. ❹ The `cancellationSummary` element shows that the order is no longer cancellable.

After successfully canceling an order, and depending on the order status before the cancellation, the order will change to one of the values detailed in the following table:

Order status before cancel	Original order status following cancel
`BOOKED`	`RELEASED`
`CONFIRMED`	`CANCELED`
`TICKETED`	`WITHDRAWN`

Partial cancel

When partially canceling an order, all the Ticketable Fares for a given Leg Solution must be canceled together.

Step 1 – Find out if an order can be partially canceled

retrieveBookingRecordRequest

Before proceeding with the cancellation, Partners must first determine whether the order can be partially canceled. To do this, perform a `retrieveBookingRecordRequest` passing in the following data:

- `recordLocator`.

Example retrieveBookingRecordRequest:

```xml
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:retrieveBookingRecordRequest version="2.0">
      <com:context>
        <com:distributorCode>DEMO</com:distributorCode>
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
      <book:recordLocator>B-DEMO-EPE000291</book:recordLocator>
      <book:responseSpec>
        <book:includeOrderDetails>true</book:includeOrderDetails>
        <book:includeOrderCosts>true</book:includeOrderCosts>
        <book:includePassengerDetails>true</book:includePassengerDetails>
        <book:includeFinancials>true</book:includeFinancials>
        <book:includePaymentRequirements>true</book:includePaymentRequirements>
        <book:includeTicketingOptions>true</book:includeTicketingOptions>
        <book:includeNotes>true</book:includeNotes>

<book:includeFulfillmentInformation>true</book:includeFulfillmentInformation>
      </book:responseSpec>
    </book:retrieveBookingRecordRequest>
  </soap:Body>
</soap:Envelope>
```

retrieveBookingRecordResponse

In the `retrieveBookingRecordResponse`, SilverCore returns a `cancellationSummary` container indicating whether a partial cancellation is allowed for the order and, if so, what penalties apply.

An order is eligible for partial cancellation if `isPartiallyCancellable= "true"` is returned in the `cancellationSummary` container element:

```xml
<cancellationSummary isCancellable="true" isPartiallyCancellable="true">
```

Example retrieveBookingRecordResponse:

```xml
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:retrieveBookingRecordRequest version="2.0">
      <com:context>
```

```
        <com:distributorCode>DEMO</com:distributorCode>
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
      <book:recordLocator>B-DEMO-EPE000291</book:recordLocator>
      <book:responseSpec>
        <book:includeOrderDetails>true</book:includeOrderDetails>
        <book:includeOrderCosts>true</book:includeOrderCosts>
        <book:includePassengerDetails>true</book:includePassengerDetails>
        <book:includeFinancials>true</book:includeFinancials>
        <book:includePaymentRequirements>true</book:includePaymentRequirements>
        <book:includeTicketingOptions>true</book:includeTicketingOptions>
        <book:includeNotes>true</book:includeNotes>

<book:includeFulfillmentInformation>true</book:includeFulfillmentInformation>
      </book:responseSpec>
    </book:retrieveBookingRecordRequest>
  </soap:Body>
</soap:Envelope>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
      <ns2:retrieveBookingRecordResponse
conversationToken="SplitTicketing_Shop_Validate_Book_Demo"
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
         <requestStatus systemId="XGjTZO">
            <success>true</success>
         </requestStatus>
         <ns2:bookingRecord recordLocator="B-VIRGINTT-EPE000291" status="DEBIT">
            <updateable>
               <passengerIdentity>true</passengerIdentity>
               <passengerAddress>true</passengerAddress>
               <passengerContactInfo>true</passengerContactInfo>
               <loyaltyCard>true</loyaltyCard>
               <travelDocument>true</travelDocument>
            </updateable>
            <bookingDate>2023-06-29Z</bookingDate>
            <departureDate>2023-07-04</departureDate>
            <numberOfOrders>1</numberOfOrders>
            <revenueTotal currency="GBP">44.20</revenueTotal>
            <receiptsTotal currency="GBP">0.00</receiptsTotal>
            <passengers>...</passengers>
            <orders>
               <order orderID="O-VIRGINTT-EPE000291-EXF000001" status="BOOKED">
                  <dateBooked>2023-06-29Z</dateBooked>
                  <holdExpiration>2023-06-29T13:20:44Z</holdExpiration>
                  <refundEligibility eligible="true"
cancelRequiredForRefund="true">
```

```
                        <penalty currency="GBP">0.00</penalty>
                     </refundEligibility>
❶                    <cancellationSummary isCancellable="true"
isPartiallyCancellable="true">
                        <cancellationOptions>
                           <cancellationOption>
                              <refundTarget type="FORM_OF_PAYMENT"/>
                              <penalty>
                                 <total currency="GBP">0.00</total>
                              </penalty>
                           </cancellationOption>
                        </cancellationOptions>
                        <priceReversals>
                           <total currency="GBP">44.20</total>
                           <breakdown>
                              <component type="Product_Sale">
                                 <amount currency="GBP">22.10</amount>
                              </component>
                              <component type="Product_Sale">
                                 <amount currency="GBP">22.10</amount>
                              </component>
                           </breakdown>
                        </priceReversals>
                     </cancellationSummary>
                     ...
❷                    <combinedTicketableFareSets>
                        <combinedTicketableFareSet
combinedTicketableFareSetId="CTF_0">
                           <combinedTicketableFareLocators>
                              <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
0</combinedTicketableFareLocator>
                              <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
1</combinedTicketableFareLocator>
                           </combinedTicketableFareLocators>
                        </combinedTicketableFareSet>
                        <combinedTicketableFareSet
combinedTicketableFareSetId="CTF_1">
                           <combinedTicketableFareLocators>
                              <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
2</combinedTicketableFareLocator>
                              <combinedTicketableFareLocator>TF-EPE000291-EXF000001-
3</combinedTicketableFareLocator>
                           </combinedTicketableFareLocators>
                        </combinedTicketableFareSet>
                     </combinedTicketableFareSets>
                  </order>
               </orders>
               ...
            </ns2:bookingRecord>
         </ns2:retrieveBookingRecordResponse>
      </soap:Body>
   </soap:Envelope>
```

❶ The order can be partially canceled. ❷ The `combinedTicketableFareSets` container returns the Ticketable Fares that must be canceled together. In this round-trip example, Ticketable Fares with the Id ending in 0 and 1 must be canceled to cancel the outbound leg. The Ticketable Fares with the Id ending in 2 and 3 must be canceled together to cancel the return leg.

Step 2 – Find out if the Ticketable Fares can be canceled

retrieveCancellationSummaryRequest

Before canceling the Ticketable Fares on the order, Partners must perform a `retrieveCancellationSummaryRequest` to determine if they can be canceled, passing in the following data:

- `recordLocator` and `orderLocator`.
- `ticketableFareLocators` containing the Id of each Ticketable Fare on the order you wish to cancel.

Example request for the outbound Leg Solution for a return journey:

```
<soap:Envelope xmlns:com= "http://railgds.net/ws/commontypes"
               xmlns:book="http://railgds.net/ws/booking"
               xmlns:shop="http://railgds.net/ws/shopping"
               xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Header/>
   <soap:Body>
      <book:retrieveCancellationSummaryRequest version="2.0">
         <com:context>
            <com:distributorCode>DEMO</com:distributorCode>
            <com:pointOfSaleCode>GB</com:pointOfSaleCode>
            <com:channelCode>IOS</com:channelCode>
         </com:context>
         <book:recordLocator>B-DEMO-YLB0000TI</book:recordLocator>
         <book:orderLocator>O-DEMO-YLB0000TI-FTY000001</book:orderLocator>
❶          <book:ticketableFareLocators>
            <book:ticketableFareLocator>TF-YLB0000TI-FTY000001-
0</book:ticketableFareLocator>
            <book:ticketableFareLocator>TF-YLB0000TI-FTY000001-
1</book:ticketableFareLocator>
           </book:ticketableFareLocators>
      </book:retrieveCancellationSummaryRequest>
   </soap:Body>
</soap:Envelope>
```

❶ The `ticketableFareLocators` container element contains the Ids of each Ticketable Fare for the chosen Leg Solution.

retrieveCancellationSummaryResponse

In the response message, SilverCore returns a `cancellationSummary` container indicating whether the Ticketable Fares for the Leg Solution can be canceled and, if so, what penalties apply.

The Ticketable Fares can be canceled if `isCancellable="true"` is returned in the `cancellationSummary` container element, as shown in the following example:

```
<cancellationSummary isCancellable="true" isPartiallyCancellable="false">
```

Example retriveCancellationSummaryResponse:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns2:retrieveCancellationSummaryResponse
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
      <requestStatus systemId="CXVN1B">
        <success>true</success>
      </requestStatus>
      <ns2:orders>
        <ns2:order orderID="O-DEMO-YLB0000TI-FTY000001">
          <ns2:cancellationSummary isCancellable="true"
isPartiallyCancellable="false">
            <cancellationOptions>
              <cancellationOption>
                <refundTarget type="FORM_OF_PAYMENT"/>
                <penalty>
                  <total currency="GBP">0.00</total>
                </penalty>
              </cancellationOption>
            </cancellationOptions>
          </ns2:cancellationSummary>
          <ns2:voidSummary isVoidable= "false"/>
        </ns2:order>
      </ns2:orders>
    </ns2:retrieveCancellationSummaryResponse>
  </soap:Body>
</soap:Envelope>
```

Step 3 – Cancel the Ticketable Fares

cancelBookingRecordRequest

To partially cancel a booking, perform a `cancelBookingRecordRequest`, passing in the following data:

- `recordLocator` and `orderLocator`.
- `ticketableFareLocator` containing the Id of each Ticketable Fare on the order you wish to cancel.
- `expectedCancellationFee`, which must match the penalty total returned in the `retrieveCancellationSummaryResponse`.

Example cancelBookingRecordRequest:

```
<soap:Envelope xmlns:com="http://railgds.net/ws/commontypes"
xmlns:book="http://railgds.net/ws/booking"
xmlns:shop="http://railgds.net/ws/shopping"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <book:cancelBookingRecordRequest version="2.0">
      <com:context>
        <com:distributorCode>DEMO</com:distributorCode>
        <com:pointOfSaleCode>GB</com:pointOfSaleCode>
        <com:channelCode>IOS</com:channelCode>
      </com:context>
      <book:recordLocator>B-DEMO-RKU0000BS</book:recordLocator>
      <book:orderLocator>O-DEMO-VWP0000UA-UHW000001</book:orderLocator>
      <book:ticketableFareLocators>
        <book:ticketableFareLocator>TF-YLB0000TI-FTY000001-
0</book:ticketableFareLocator>
            <book:ticketableFareLocator>TF-YLB0000TI-FTY000001-
1</book:ticketableFareLocator>
      </book:ticketableFareLocators>
      <book:expectedCancellationFee
currency="GBP">0.00</book:expectedCancellationFee>
      <book:responseSpec>
        <book:returnReservationDetails>true</book:returnReservationDetails>
      </book:responseSpec>
    </book:cancelBookingRecordRequest>
  </soap:Body>
</soap:Envelope>
```

cancelBookingRecordResponse

As with a full order cancel, the resulting `cancelBookingRecordResponse` shows the success/fail status for the partial cancellation and optionally echoes back a complete Booking Record containing passenger information, Travel Segments, and Ticketable Fares, along with full payment status and financials. It also includes information about any refund due.

After successfully partially canceling an order, the original order will be cloned into a new order. Observe the following changes to the status of the original order and the status of the cloned order:

Order status before cancel	Original order status following cancel	Cloned order status following cancel
`BOOKED`	`REPLACED`	`BOOKED`
`CONFIRMED`	`REPLACED`	`CONFIRMED`
`TICKETED`	`REPLACED`	`CONFIRMED`

Info

For E-Tickets, Partners must send a claimValueDocumentRequest to issue the necessary documents that entitle the bearer to travel.

Example cancelBookingRecordResponse:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
      <ns2:cancelBookingRecordResponse
xmlns:ns8="http://railgds.net/ws/extractbooking"
xmlns:ns7="http://railgds.net/ws/vault" xmlns:ns6="http://railgds.net/ws/search"
xmlns:ns5="http://railgds.net/ws/onaccount"
xmlns:ns4="http://railgds.net/ws/keystore"
xmlns:ns3="http://railgds.net/ws/shopping"
xmlns:ns2="http://railgds.net/ws/booking"
xmlns="http://railgds.net/ws/commontypes">
         <requestStatus systemId="Cksk9Y">
            <success>true</success>
         </requestStatus>
❶         <ns2:bookingRecord recordLocator="B-DEMO-PEW00016M" status="CREDIT">
            <bookingDate>2023-06-15Z</bookingDate>
            <departureDate>2023-06-22</departureDate>
            <numberOfOrders>2</numberOfOrders>
            <revenueTotal currency="GBP">36.80</revenueTotal>
            <receiptsTotal currency="GBP">44.20</receiptsTotal>
            <passengers>...</passengers>
            <orders>
❷             <order orderID="O-DEMO-PEW00016M-XAG000001" status="CONFIRMED"
originalOrderID="O-DEMO-PEW00016M-JSC000001">
               ...
```

```
❸                <cancellationSummary isCancellable="true"
isPartiallyCancellable="false">
                      <cancellationOptions>
                         <cancellationOption>
                            <refundTarget type="FORM_OF_PAYMENT"/>
                            <penalty>
                               <total currency="GBP">14.70</total>
                               <breakdown>
                                  <component type="TICKET"
currency="GBP">14.70</component>
                               </breakdown>
                            </penalty>
                         </cancellationOption>
                      </cancellationOptions>
                      <priceReversals>
                         <total currency="GBP">22.10</total>
                         <breakdown>
                            <component type="Product_Sale">
                               <amount currency="GBP">4.70</amount>
                            </component>
                            <component type="Product_Sale">
                               <amount currency="GBP">17.40</amount>
                            </component>
                         </breakdown>
                      </priceReversals>
                   </cancellationSummary>
                   ...
                   </order>
❹               <order orderID="O-DEMO-PEW00016M-JSC000001" status="REPLACED">
                   ...
                  <cancellationSummary isCancellable="false"
isPartiallyCancellable="false"/>
                   <voidSummary isVoidable="false" isVoidableException="false"/>
                   <cancellationReason>PARTIAL_TICKET_NOT_REQD</cancellationReason>

<internalCancellationReason>PARTIAL_CANCEL</internalCancellationReason>
                   <isInventoryCanceled>true</isInventoryCanceled>
                   ...
                   </order>
            </orders>
            <paymentRequirements>...</paymentRequirements>
            <serviceFeeAllowed>true</serviceFeeAllowed>

<creditCardSurchargeFeeApplicable>false</creditCardSurchargeFeeApplicable>
            <financials>...</financials>
         </ns2:bookingRecord>
      </ns2:cancelBookingRecordResponse>
   </soap:Body>
</soap:Envelope>
```

❶ The cancellation results in a refund being due to the customer. When a refund is due, the `receiptsTotal` will be greater than the `revenueTotal`. The difference is the amount to be

refunded by Partners through a refundBookingRecordRequest. ❷ The cloned order with the remaining Ticketable Fares on the order in status `CONFIRMED`. ❸ The active order can be canceled in full `isCancellable="true"`. ❹ The original order in status `REPLACED`, which can no longer be canceled. The `internalCancellationReason` is `PARTIAL_CANCEL`.

Possible errors encountered when partially canceling Split Tickets

BK00186

SilverCore will return a `BK00186` booking error message when an attempt is made to cancel Ticketable Fares that cannot be canceled individually.

Example cancel booking error message:

```
<statusMessages>
  <statusMessage>
    <message>Must specify all ticketable fares for all canceling travel
segments</message>
    <code>BK00186</code>
  </statusMessage>
</statusMessages>
```

BK00166

SilverCore will return a `BK00166` booking error message when an attempt is made to perform a full cancel passing in all the Ticketable Fares on an order.

Example cancel booking error message:

```
<statusMessages>
  <statusMessage>
    <message>All ticketable fares on an order can not be specified</message>
    <code>BK00166</code>
  </statusMessage>
</statusMessages>
```